Bizora

Your Business, Simplified.

Project Portfolio

A local-first business management and billing desktop app for small businesses

Electron · Next.js · TypeScript · SQLite · Drizzle ORM · Tailwind · shadcn/ui

Prepared by

Abhishek Yadav

Software Engineer · Supporting document: personal project

STATUS

In active development

PROJECT SUMMARY

1 Overview

Small businesses need billing, customer management, inventory tracking and reports, but running an online application brings hosting, database and maintenance costs they do not need. **Bizora runs locally on the business owner's computer.** Business data and core operations stay available without a separately hosted backend or online database. The platform is also **country-aware**: each business configures its own tax, food-safety and regulatory information for its location.

12	0	100%	11
feature areas	servers to manage	local business data	roadmap phases

Install Bizora → Set up your business → Start managing your business.

The problem and the approach

The problem	How Bizora addresses it
Hosting, database and maintenance costs for a small shop	Local SQLite database on the owner's computer: no monthly hosting, no external database
Complex deployment and technical setup	Packaged desktop app, guided first-launch wizard, automated application setup
Tools hard-coded for one country's tax rules	Configurable tax and regulatory models: GST (India), GST (New Zealand) and more later
Billing, stock, expenses and staff scattered across tools	One app covering customers, products, billing, purchases, expenses, workers and reports
Fear of losing data or breaking it on updates	Safety backup before migrations, versioned Drizzle migrations, backup and restore

Why it is useful for small companies

- **Cost:** no server, no hosting subscription, no database licence. Core features work without internet.
- **Simplicity:** an owner can install it and start without technical knowledge; navigation is limited to daily operations.
- **Control:** the business data lives on the owner's own computer, fast to access and under their control.
- **Fit:** it begins with small dairy businesses (milk, curd, tofu, paneer) but the product model is generic enough for groceries, bakeries, cafes, restaurants, small retailers and wholesalers.
- **Compliance-ready:** tax numbers, food-safety licences and other registrations are configured per country, not hard-coded.

Project goal

Give small businesses professional business-management tools without forcing them to manage servers, databases, hosting or complicated software infrastructure.

WHAT THE APP COVERS

2 Features at a glance

01 Country-aware setup
Country, tax and regulatory details configured per business

02 Customers
Person and business customers with full history

03 Products
Products, units, pricing and inventory

04 Daily entries
Record daily transactions quickly, multi-item

05 Billing
Invoices, monthly statements, print and PDF

06 Stock purchases
Track purchases and how stock enters the business

07 Suppliers
Manage who you buy from, with history

08 Expenses
Operating costs by your own categories

09 Workers
Lightweight worker records and payments

10 Payments
Paid, unpaid, partial, outstanding balances

11 Reports
Sales, purchases, expenses, profit overview

12 Backup
Local data with backup and restore

Payment details on bills

Businesses configure payment information once (bank name, account name and number, branch, routing / bank code, payment identifier, payment QR and instructions) and show it on invoices. **No payment gateway is required**: customers scan the QR code or use the bank details. Payment details are **snapshotted on each invoice**, so changing them later never alters invoices already generated.

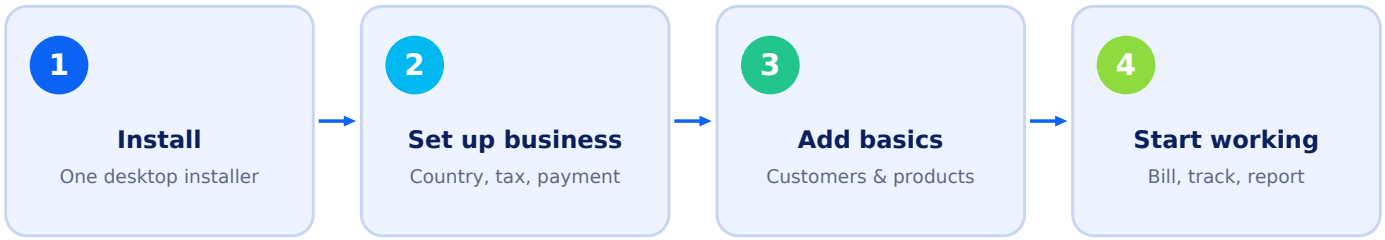
Customer, product and daily-entry capabilities

Area	Capabilities
Individual customer	First and last name, phone, email, address and location details, transaction history
Business customer	Business name, contact person, phone, email, tax registration details, business address, business transaction history
Products	Products, categories, units, pricing, opening stock and stock tracking
Daily entries	Select a customer, add multiple products, set quantities, calculate totals, apply discounts, track payment status, view previous entries
Billing	Invoice numbering, customer and product details, tax calculation, discounts, payment status; print or export PDF
Payment tracking	Paid, unpaid and partially paid status, outstanding balance, customer payment history, mark invoice as paid

DESIGNED FOR NON-TECHNICAL OWNERS

3 Built for customer simplicity

Bizora is packaged as a desktop application that can be installed and used with minimal technical setup. The first launch is a guided wizard; day-to-day use is a short list of operational screens.



Owners never have to…

- ✕ Configure a server
- ✕ Install a database
- ✕ Type terminal commands
- ✕ Handle deployment

Owners simply…

- ✓ Install one desktop app
- ✓ Follow a guided setup wizard
- ✓ Use plain-language screens
- ✓ Print or export invoices as PDF

Navigation stays focused on operations

Main navigation	Settings (configuration lives here)
Dashboard · Daily Entries · Customers · Products · Stock Purchases · Suppliers · Expenses · Workers · Reports · Settings	Business · Tax & Registrations · Payment Details · Invoice & Billing · Expense Categories · Product Categories · Data & Backup · Application

Quick Sale: no account, no product picking

Dairy shops, sweet shops and counters often serve unknown customers. The operator just enters an amount and a payment method. Behind the scenes Bizora records the sale against a Walk-in Customer and a Quick Sale product, so the database keeps its required relationships while the owner never creates a customer or selects a product for a small counter sale.



The dashboard answers one question

How is my business doing today? Planned widgets: today's sales and entries, outstanding amount, recent transactions, sales overview, top products, recent customers, low-stock items, recent purchases and expenses, worker payments, and revenue vs expenses.

ONE CORE APP, MANY COUNTRIES

4 Country-aware by design

Bizora does not hard-code country-specific requirements such as FSSAI or GSTIN. During setup the business selects its country and configures the regulatory information relevant to it. A new country or regulatory system needs **no product change**.

Setup captures

Country, business name, logo, address, city, state / region, postal code, contact information, business type, tax information, food-safety information and other business registrations.

Configurable tax information

`Tax System` · `Tax Registration Number` · `Tax Registered` · `Tax Rate`

Country	Tax system	Registration label
India	GST	GSTIN
New Zealand	GST	GST Number
Future countries	Additional tax systems supported by the same model	Configurable

Generic regulatory and food-safety structure

Instead of a column such as `fssai_number`, Bizora stores each registration as a generic record:

`Authority / Department` · `Registration Type` · `Registration Number` · `Issue Date` · `Expiry Date`

Field	Example: Indian food business	Example: New Zealand business
Authority	FSSAI	Whichever authority applies
Registration type	Food Business Licence	Configured by the business
Registration number	XXXXXXXXXXXXXX	Configured by the business

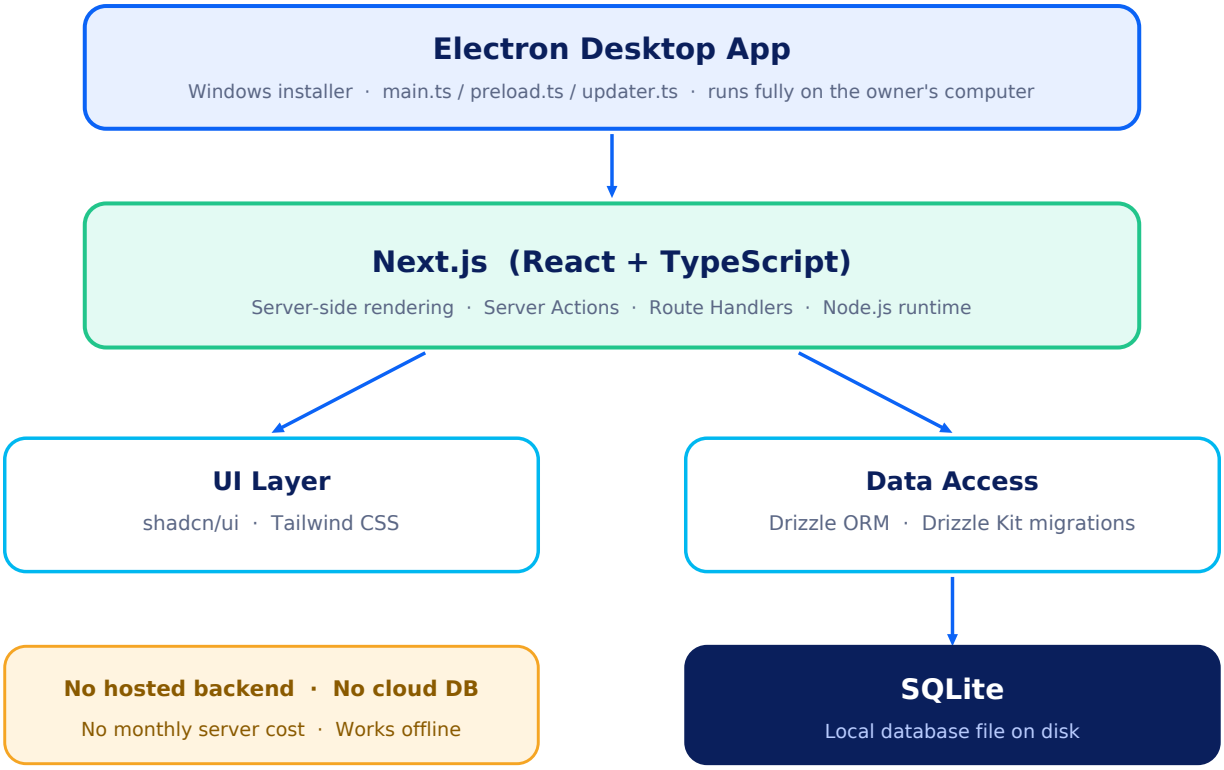
Why this matters (engineering view)

- No columns like `fssai_number`, `gst_number`, `nzbn_number` or `abn_number`; one extensible `business_registrations` table with a `metadata` field instead.
- Tax data is stored as configurable tax registrations, not country-specific columns.
- The UI shows locally relevant terminology (GSTIN vs GST Number) from configuration, so the same code serves every country.
- Regulatory screens in setup are shown based on the selected country and business type.

SINGLE APP, SINGLE REPOSITORY

5 Architecture

Bizora is one application in one repository. Electron wraps a Next.js app whose server layer talks to a local SQLite database through Drizzle ORM. There is no separately hosted backend.



Tech stack

Layer	Technology
Frontend	Next.js, React, TypeScript, Tailwind CSS, shadcn/ui
Desktop	Electron (Windows installer first; macOS and Linux planned)
Backend / server	Next.js Server Actions, Next.js Route Handlers, Node.js runtime
Database	SQLite, Drizzle ORM, Drizzle Kit (versioned migrations)
Development	TypeScript, ESLint, Git, GitHub

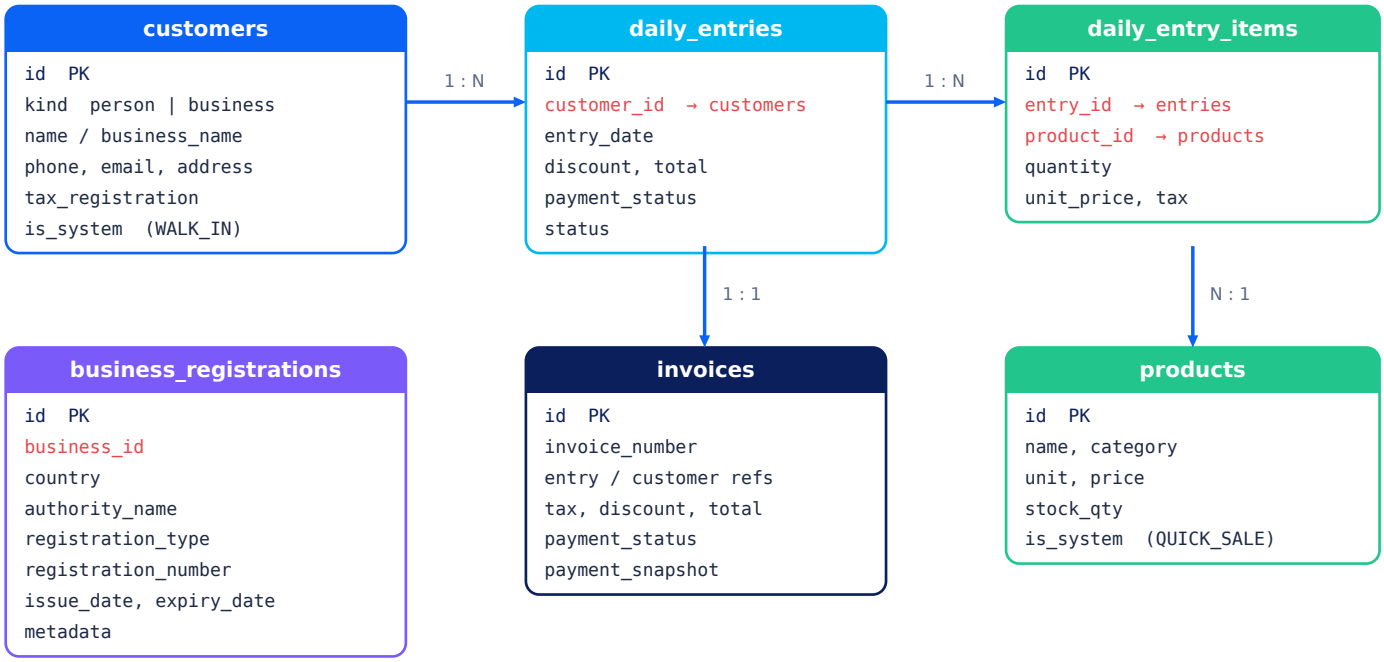
Architecture principles

Single repository	**Single desktop application**	**Local-first**
No separately hosted backend	Local SQLite database	Type-safe database access
Server-side rendering	Component-based UI	Automated application setup
Country-aware configuration	Extensible tax and regulatory models	Backward-aware database versions

RELATIONAL, EXPLICIT, SAFE

6 Data model and system records

Operational data is relational and explicit. Normal entries keep required references, and these relationships are intentionally **not nullable**. *Conceptual model; fields illustrate the design and may evolve.*



System records

Setup creates two application-owned records. Quick Sale uses these records instead of skipping required relationships.

Record	Type	Purpose
WALK_IN_CUSTOMER	Customer	Counter sales to unknown customers
QUICK_SALE	Product	Quick counter sales (quantity 1)

	Normal records	System records
Customers	Visible in customer management	Hidden from normal management
Products	Visible in product management	Hidden from normal management

- System records are filtered out of lists, normal selectors and customer-facing ledgers where a real customer is expected.
- **Transaction history rule:** once a sale is recorded it is business history. UI visibility rules must never remove the underlying transaction from sales reports.
- They are not part of the wizard's customer or product inputs, because they are not user-created business records.

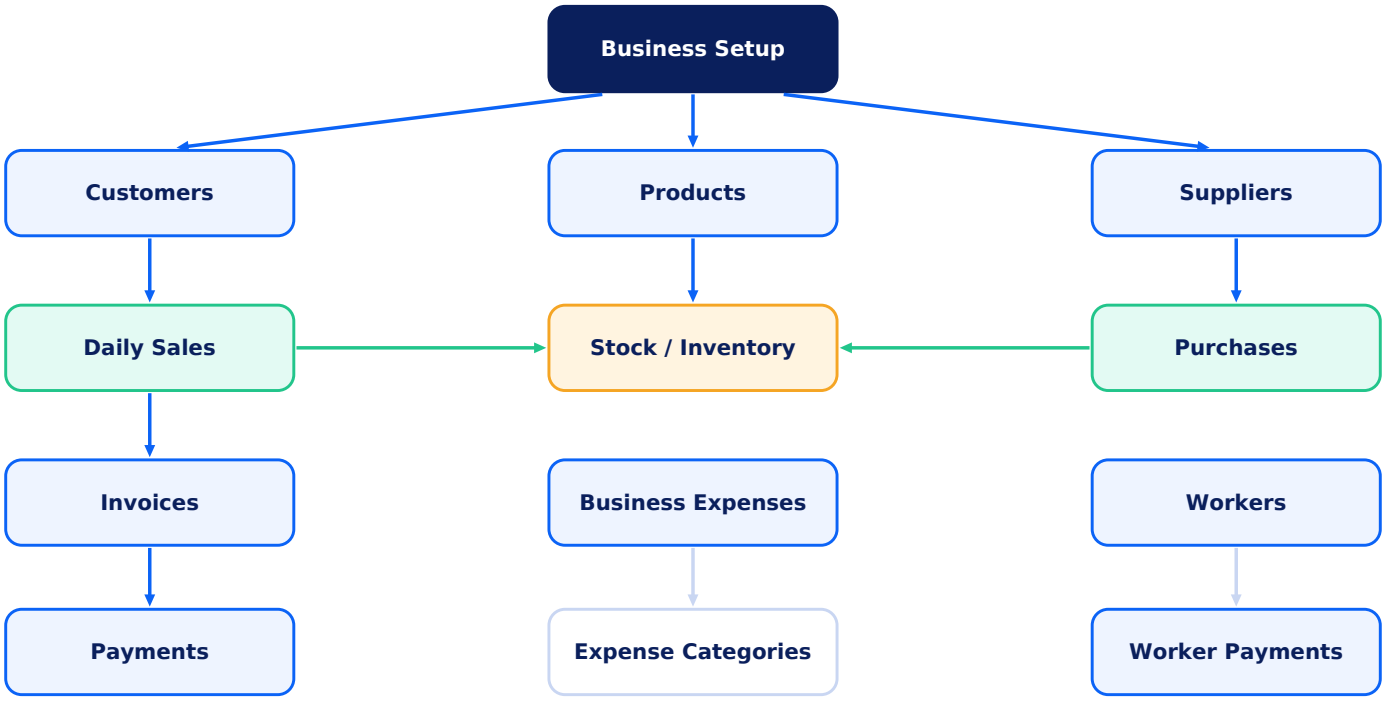
Project structure

```
bizora/
├── app/          login · setup · dashboard · daily-entries · customers · products
│                 stock-purchases · suppliers · expenses · workers · reports
├── components/ ui · layout · dashboard · customers · products · billing · reports
├── lib/          db · auth · validations · utilities
├── electron/     main.ts · preload.ts · updater.ts
├── drizzle/      migrations/
├── scripts/      setup.ts · migrate.ts · backup.ts
└── public/platform-assets/   logos and app icons
```

THE FULL CYCLE OF A SMALL BUSINESS

7 Business workflows

Bizora follows the day-to-day cycle: set up the business, record sales and purchases, invoice, collect payment, track costs and read reports.



sales · purchases · expenses · worker payments → profit overview

Reports & Dashboard — "How is my business doing today?"

Sales → Revenue | Purchases → Inventory | Expenses → Operating cost | Worker pay → Labour cost

Daily entry

Select customer → Select products → Enter quantities → Calculate total → Set payment status → Save entry

Billing

Customer → Daily entries → Create bill (tax, discount) → Print / PDF → Mark as paid

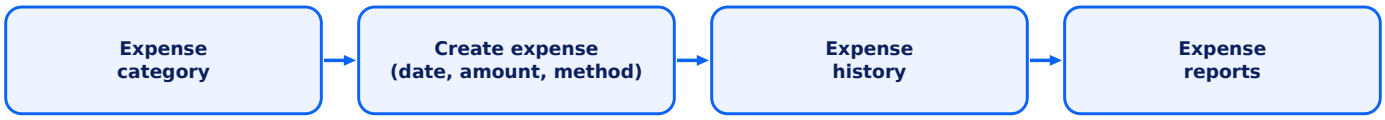
Purchasing, expenses and workers

Stock purchase. Purchases are recorded separately from ordinary expenses because they can increase inventory. One purchase can contain multiple products.

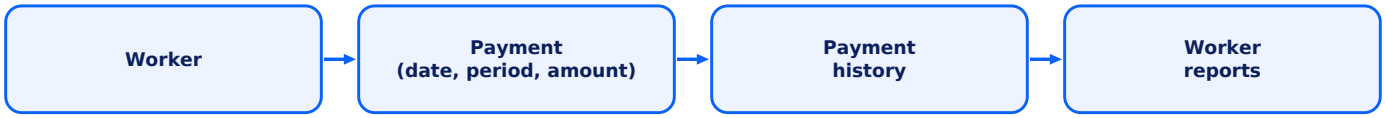



```
15 September 2026   Supplier: ABC Foods
  Milk 50 litres · Paneer 20 kg · Curd 30 kg      Purchase Total: ₹12,000
```

Expenses. Costs that are not stock purchases (rent, electricity, transport, repairs, maintenance, packaging, equipment). Businesses maintain their own categories.



Worker payments. Lightweight worker records (name, contact, role, joining date, payment type and default amount, active status) without complex HR features. Worker payments are tracked separately from general expenses, so the owner can see what was paid to workers in any week or month.



Customer ledger. Each customer page shows information, transaction history, outstanding balance, create-bill action and payment history.

Supplier management

Supplier name, contact person, phone, email, tax registration number, address, notes and active / inactive status. Supplier history can later reveal purchasing activity and spending.

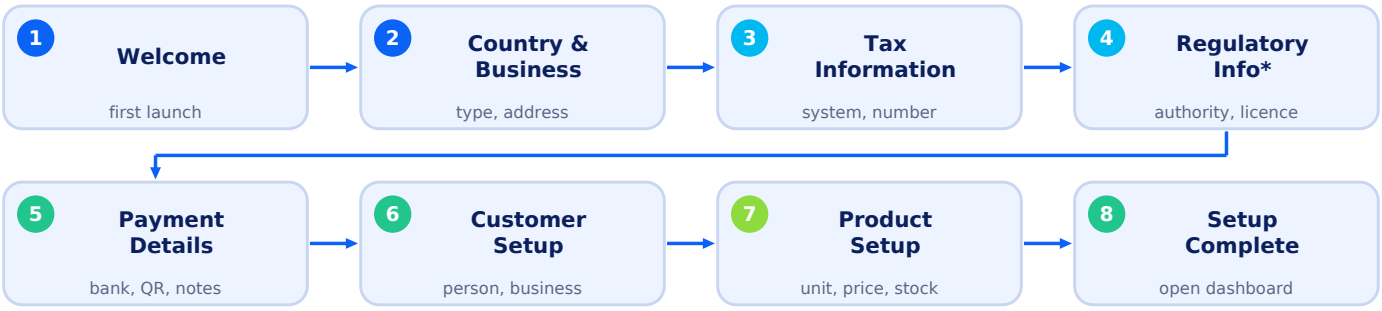
Planned reports

Sales	Costs	Overview
Daily and monthly sales · total revenue · outstanding payments · customer and product sales · payment reports	Stock purchase and supplier reports · expense reports · worker payment reports	Business overview · revenue vs expenses · purchase vs sales · profit overview

FROM INSTALL TO DASHBOARD

8 First-launch setup wizard

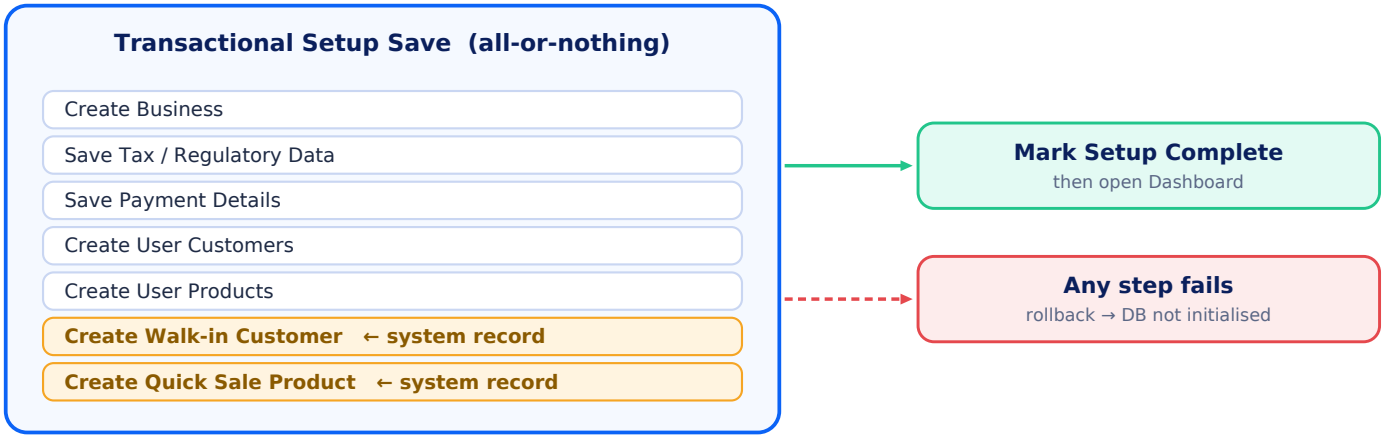
The first launch is a setup wizard that collects the business configuration and the user's own customers and products.



* Regulatory information is shown based on the selected country and business type.

Transactional finalisation

Finalisation is handled by **one setup persistence layer** and is transactional: the database is not considered initialised until the whole operation succeeds. The two system records are created in the same transaction.

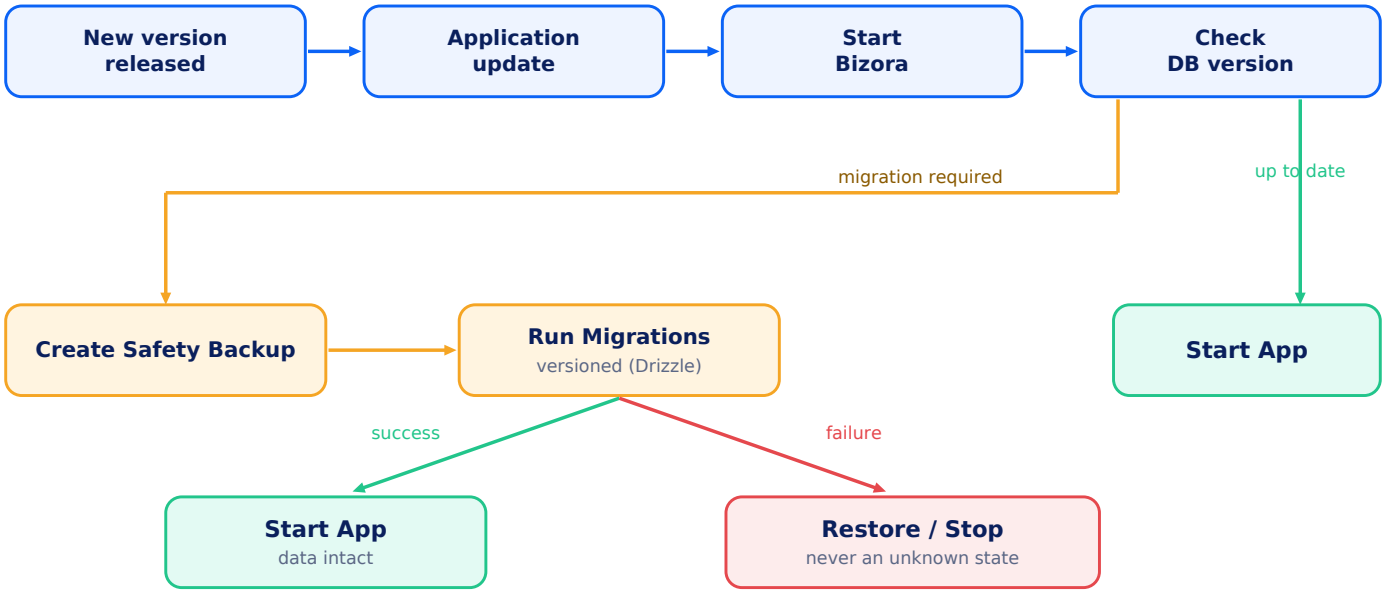


- If any step fails, nothing is committed, so the app never starts from a half-initialised database.
- Setup creates the Walk-in Customer and Quick Sale product exactly once for an initialised business.
- Payment details collected here (bank, identifier, QR, instructions) become the defaults snapshotted onto invoices.

NEVER DESTROY BUSINESS DATA

9 Updates, migrations and data safety

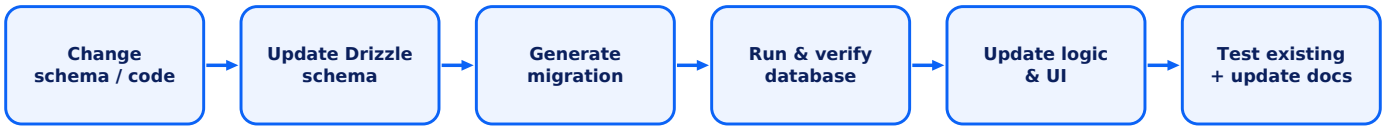
Application updates and business-data migrations are separate concerns. **Updating the application must never destroy or silently replace the business database.**



Rules

- The local SQLite database is persistent business data; application binaries may be replaced during an update.
- Migrations are versioned through Drizzle and never assume data can be discarded.
- A safety backup is created before destructive or structural migrations where appropriate.
- A failed migration must not leave the app in an unknown state: restore or stop.
- New features must be backward-aware of existing database versions.
- System records are part of the data model: migrations must not create duplicate `WALK_IN_CUSTOMER` or `QUICK_SALE` records for an already initialised business.

Development workflow



Local-first data and privacy

The core business database is stored on the user's computer; no cloud database or hosted backend is needed for core functionality. Planned: manual backup, automatic backups, backup history, restore from backup and data export. Because data is local, users are responsible for maintaining backups.

SMALL COMPANIES AND TECH PLANNING

10 Usage: who benefits and how

Bizora starts with workflows for small dairy businesses while keeping the architecture generic. The table shows how the same building blocks map to different small businesses.

Business type	How Bizora helps
Dairy and sweet shops (starting point)	Quick Sale for counter customers, daily entries for regular customers, monthly statements, stock purchases of milk, paneer, curd, tofu, FSSAI licence details on invoices
Grocery and small retailers	Product catalogue with units and opening stock, supplier purchases that update inventory, low-stock overview, tax-aware invoices
Bakeries and cafes	Fast counter sales, packaging and equipment expenses, worker payments by week or month, daily sales vs cost overview
Restaurants	Multi-item entries, discounts, expense categories, regulatory registration tracking with expiry dates
Wholesalers	Business customers with tax registrations, outstanding balances, supplier history, purchase vs sales reports
Businesses in other countries	Pick a country, configure tax system and registrations; no code change needed

Technical planning approach

Concern	Decision
Deployment	Electron-packaged desktop app, automated setup, no terminal commands for the owner
Persistence	Embedded SQLite file, Drizzle schema as the single source of truth, generated migrations
Server logic	Next.js Server Actions and Route Handlers running inside the desktop app
Extensibility	Generic registration, tax and category models instead of country or business-type columns
Safety	Transactional setup, non-null relations, system records, backup before migration
Maintainability	Single repo, TypeScript end to end, schema and code updated together, docs updated with each change

What this project demonstrates

- Full-stack TypeScript across UI, server logic and database, shipped as a desktop application.
- Domain modelling for real-world variation (country-aware tax and regulation) without forking the codebase.
- Data integrity thinking: transactional writes, explicit relations, snapshotting invoice data, migration safety.
- Product thinking: a non-technical user can install it and start, and the scope is deliberately kept lightweight.

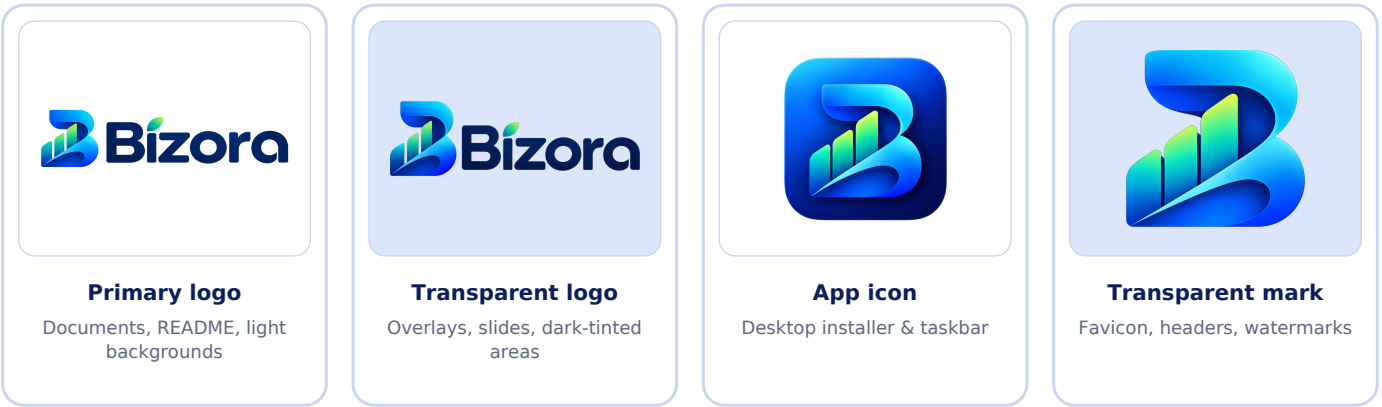
WHERE THE PROJECT IS HEADING

11 Roadmap and brand assets

Bizora is under active development. Items below are planned work; the architecture, schema, UI and feature set may change as the project evolves.

Phase	Focus	Planned scope
1	Foundation	Project init, Next.js, Electron, TypeScript, Tailwind, shadcn/ui, SQLite, Drizzle, migrations
2	Business setup	First-launch detection, country, business type, wizard, logo upload, tax and regulatory configuration, settings
3	Customers	Person and business customers, list, details, history, outstanding balance
4	Products	Categories, units, pricing, opening stock, stock tracking, system product
5	Daily operations	Daily entries and items, payment status, customer ledger, Quick Sale, walk-in workflow
6	Billing	Invoice numbering, preview, PDF, printing, mark as paid
7	Purchases and suppliers	Supplier management, purchases and items, payment tracking, stock updates, history
8	Expenses	Categories, expenses, payment tracking, history, reports
9	Workers	Worker records, payment tracking and history, reports
10	Reports	Sales, revenue, outstanding, customer, product, purchase, supplier, expense, worker, profit overview
11	Desktop production	Windows installer, automatic setup, DB initialisation, migrations, updates, backup and restore, logging
Future	Growth	macOS and Linux, thermal printers, automatic backups, data export, more business types, advanced reporting, more countries

Brand assets and usage



Primary logo
Documents, README, light backgrounds

Transparent logo
Overlays, slides, dark-tinted areas

App icon
Desktop installer & taskbar

Transparent mark
Favicon, headers, watermarks

Use the primary logo on white or light backgrounds and the transparent versions on tinted or coloured areas. The app icon is for the installer and taskbar; the transparent mark is for headers, favicons and watermarks. Keep clear space around the logo and do not stretch or recolour it.